# Tarea 2: Programación Binaria y Entera

## Formulaciones matemáticas e implementación en código

Maximiliano Vaca Montejano

Centro de Investigación en Matemáticas A.C.
Métodos de Optimización I • Aguascalientes

Octubre, 2026

### Resumen

*Este documento organiza las respuestas a los diez ejercicios de la Tarea 2 Programación Binaria y Entera. Cada ejercicio cuenta con apartados para los datos y supuestos, la formulación matemática, la implementación en código y la interpretación de resultados.*

***Palabras clave:*** *programación binaria, programación entera, formulación matemática, optimización, CPLEX, DOcplex.*

## Índice

**1. Asignación de órdenes a máquinas** **3**

**2. Asignación de personal** **6**

**3. Planeación de horarios escolares** **9**

**4. El camino más corto** **13**

**5. Asignación de carteros** **16**

**6. Distribución de productos** **19**

**7. Contratación de personal** **21**

**8. Planeación de encuestas** **24**

**9. Planeación de transporte** **26**

**10. Personal de obra** **28**

**Material de la tarea** **31**

# 1. Asignación de órdenes a máquinas

## 1.1. Datos y supuestos

- Cinco **órdenes** $(A_1, A_2, A_3, A_4, A_5)$.
- Cinco **máquinas** $(M_1, M_2, M_3, M_4, M_5)$.
- Cada máquina puede realizar cada tarea con el costo indicado en la Tabla 1 del enunciado.
- Determinar la asignación óptima que **minimice el costo total** de completar las órdenes.
- **Cada máquina solo puede procesar una orden**.
- **Todas las órdenes deben ser atendidas**.

## 1.2. Formulación matemática

### Variables de decisión

Variables binarias; ¿La orden $i$ se atiende en qué máquina $j$?

$$x_{ij} = \begin{cases} 1 & \text{si se atiende la orden } i \text{ en la máquina } j, \\ 0 & \text{si no,} \end{cases}$$

donde $i \in \{0, 1, 2, 3, 4\} = \{A_1, A_2, A_3, A_4, A_5\}$ y $j \in \{0, 1, 2, 3, 4\} = \{M_1, M_2, M_3, M_4, M_5\}$.

### Restricciones

**Restricción 1: Procesamiento por máquina.** Cada máquina solo puede procesar una orden. Aquí $j$ está fijo (una máquina específica) y sumamos sobre todas las órdenes $i$:

$$\sum_i x_{ij} = 1 \quad \forall\, j.$$

**Restricción 2: Atención de órdenes.** Todas las órdenes deben ser atendidas. Aquí $i$ está fijo (una orden específica) y sumamos sobre todas las máquinas $j$:

$$\sum_j x_{ij} = 1 \quad \forall\, i.$$

**Condición de dominio.**

$$x_{ij} \in \{0, 1\} \quad \forall\, i, j.$$

### Función objetivo

**Para el caso a):**

$$\text{mín}\, z = \sum_i \sum_j c_{ij} x_{ij}.$$

**Para el caso b):** Suponga que procesar cada orden en cada máquina implica un costo promedio de preparación de 10 unidades. El modelo se modifica como:

$$\begin{aligned}
\text{mín } z &= \sum_i \sum_j (c_{ij} + 10) x_{ij} \\
&= \sum_i \sum_j c_{ij} x_{ij} + \sum_i \sum_j 10 x_{ij} \\
&= \sum_i \sum_j c_{ij} x_{ij} + 10 \sum_i \sum_j x_{ij} \\
&= \sum_i \sum_j c_{ij} x_{ij} + 10 \times 5 \\
&= \sum_i \sum_j c_{ij} x_{ij} + 50.
\end{aligned}$$

La asignación óptima no debería cambiar respecto al caso a) al solo sumar una constante.

### 1.3. Implementación en código

```python
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Asignacion de ordenes a maquinas')

# Tabla 1
costos = [
    # M1  M2  M3   M4    M5
    [16,   4,  9,   5,   6],  # A1
    [2,   14,  7,   5,  13],  # A2
    [8,   10,  3,  12,  11],  # A3
    [3,    7,  6,  10,   5],  # A4
    [3,    6,  8,  11,   7],  # A5
]

costos_np = np.array(costos)

# ordenes y maquinas
num_ordenes = costos_np.shape[0]  # 5 filas
num_maquinas = costos_np.shape[1] # 5 columnas

ordenes_i = range(num_ordenes)    # [0, 1, 2, 3, 4]
maquinas_j = range(num_maquinas) # [0, 1, 2, 3, 4]

x = mdl.binary_var_matrix(ordenes_i, maquinas_j, name='x')

# restricciones: cada maquina puede procesar a lo sumo una orden
mdl.add_constraints(
    mdl.sum(x[i,j] for i in ordenes_i) == 1 for j in maquinas_j
    )

# restricciones: cada orden debe ser atendida
mdl.add_constraints(
```

```
    mdl.sum(x[i,j] for j in maquinas_j) == 1 for i in ordenes_i
    )

mdl.minimize(
    mdl.sum(costos_np[i,j] * x[i,j] for i in ordenes_i for j in
    maquinas_j)
    )

# Resolver e imprimir solucion
solution = mdl.solve(log_output=True,)

print(solution)
```

**Código 1. Asignación de órdenes a máquinas.**

### 1.4. Resultados e interpretación

**Optimal objective value:** $z = 20$.

```
solution for: Asignacion de ordenes a maquinas
objective: 20
status: OPTIMAL_SOLUTION(2)
x_0_1=1
x_1_3=1
x_2_2=1
x_3_4=1
x_4_0=1
```

**Código 2. Salida del solver para el Problema 1.**

La asignación óptima consiste en asignar la Orden $A_1$ a la Máquina $M_2$, $A_2$ a $M_4$, $A_3$ a $M_3$, $A_4$ a $M_5$ y $A_5$ a $M_1$, logrando un costo total mínimo de 20 unidades.

## 2. Asignación de personal

### 2.1. Datos y supuestos

- Cinco proyectos.
- Cinco técnicos.
- Cada técnico emplea tiempos distintos.
- Tiempos en Tabla 2 del enunciado.
- Determinar la asignación que minimice el tiempo.
- Cada técnico se hace cargo de un trabajo distinto.

### 2.2. Formulación matemática

**Variables de decisión**

Qué técnico $i$ atiende determinado trabajo $j$. Variable binaria: ¿Atiende el técnico $i$ el trabajo $j$?

$$x_{ij} = \begin{cases} 1 & \text{si el técnico } i \text{ atiende el trabajo } j, \\ 0 & \text{si no,} \end{cases}$$

donde $i \in \{0,1,2,3,4\} =$ Técnico $\{1,2,3,4,5\}$ y $j \in \{0,1,2,3,4\} =$ Trabajo $\{1,2,3,4,5\}$.

**Restricciones**

**Restricción 1: Un trabajo por técnico.** Cada técnico se hace cargo de un trabajo distinto. Dejando fija la $i$ del técnico, la suma sobre todos los trabajos $j$ para ese $i$ debe ser 1:

$$\sum_j x_{ij} = 1 \quad \forall\, i.$$

**Restricción 2: Un técnico por trabajo.** Dejando fija la $j$ del trabajo, la suma sobre todos los técnicos $i$ para ese $j$ debe ser 1:

$$\sum_i x_{ij} = 1 \quad \forall\, j.$$

**Condición de dominio.**

$$x_{ij} \in \{0,1\} \quad \forall\, i,j.$$

**Función objetivo**

Minimizar el tiempo total empleado:

$$\text{mín } z = \sum_i \sum_j c_{ij} x_{ij}.$$

## 2.3. Implementación en código

```python
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Asignacion de tecnicos a trabajos')

# Tabla 2
tiempos = [
    # Tr1  Tr2  Tr3   Tr4   Tr5
    [45,   22,  30,   65,   15],  # Tec1
    [180,  163, 85,   148,  68],  # Tec2
    [21,   17,  193,  59,   65],  # Tec3
    [18,   83,  16,   70,   115], # Tec4
    [97,   75,  20,   70,   71],  # Tec5
]

tiempos_np = np.array(tiempos)

# tecnicos y trabajos
num_tecnicos = tiempos_np.shape[0]  # 5 filas
num_trabajos = tiempos_np.shape[1] # 5 columnas

tecnicos_i = range(num_tecnicos)   # [0, 1, 2, 3, 4]
trabajos_j = range(num_trabajos) # [0, 1, 2, 3, 4]

x = mdl.binary_var_matrix(tecnicos_i, trabajos_j, name='x')

# restricciones: cada tecnico i atiende exactamente un trabajo j
mdl.add_constraints(
    mdl.sum(x[i,j] for j in trabajos_j) == 1 for i in tecnicos_i
    )
# restricciones: cada trabajo j es atendido por exactamente un tecnico
    i
mdl.add_constraints(
    mdl.sum(x[i,j] for i in tecnicos_i) == 1 for j in trabajos_j
    )

mdl.minimize(
    mdl.sum(tiempos_np[i,j] * x[i,j] for i in tecnicos_i for j in
   trabajos_j)
    )

# Resolver e imprimir solucion
solution = mdl.solve(log_output=True,)

print(solution)
```

**Código 3. Asignación de personal.**

### 2.4. Resultados e interpretación

**Optimal objective value:** $z = 187$.

```
solution for: Asignacion de tecnicos a trabajos
objective: 187
status: OPTIMAL_SOLUTION(2)
x_0_1=1
x_1_4=1
x_2_3=1
x_3_0=1
x_4_2=1
```

**Código 4. Salida del solver para el Problema 2.**

El tiempo total mínimo de asignación es de 187 minutos, asignando el Técnico 1 al Trabajo 2, Técnico 2 al Trabajo 5, Técnico 3 al Trabajo 4, Técnico 4 al Trabajo 1 y Técnico 5 al Trabajo 3.

# 3. Planeación de horarios escolares

## 3.1. Datos y supuestos

- Asociar: Asignaturas – Aulas – Horas.
- 3 aulas.
- 5 horas disponibles.
- 8 asignaturas: Materias $= \{M_1, M_2, M_3, M_4, M_5, M_6, M_7, M_8\}$.
- El Docente 1 imparte 3 asignaturas: $M_1, M_2, M_8$.
- El Docente 2 imparte 5 asignaturas: $M_3, M_4, M_5, M_6, M_7$.
- El Curso 1 toma $M_1, M_2, M_3, M_4$.
- El Curso 2 toma $M_5, M_6, M_7, M_8$.

## 3.2. Formulación matemática

### Variables de decisión

Cada asignatura $i$ debe ser asignada a un aula $j$ y una hora $k$:

$$x_{ijk} = \begin{cases} 1 & \text{si la asignatura } i \text{ es impartida en el aula } j \text{ a la hora } k, \\ 0 & \text{si no,} \end{cases}$$

donde $i \in \{0, 1, \ldots, 7\} = \{M_1, \ldots, M_8\}$, $j \in \{0, 1, 2\} =$ Aula $\{1, 2, 3\}$, y $k \in \{0, 1, 2, 3, 4\} =$ Bloque $\{1, 2, 3, 4, 5\}$.

### Restricciones

**Restricción 1: Asignación única por asignatura.** Cada asignatura $i$ debe impartirse una sola vez:

$$\sum_j \sum_k x_{ijk} = 1 \quad \forall\, i.$$

**Restricción 2: Capacidad de aula y hora.** En cada aula $j$ y en cada hora $k$ se permite impartir a lo más una asignatura $i$:

$$\sum_i x_{ijk} \leq 1 \quad \forall\, j, k.$$

**Restricción 3: Restricción de docente.** Cada docente no puede impartir más de una asignatura $i$ en una misma hora $k$:

$$\sum_{i \in \{0,1,7\}} \sum_j x_{ijk} \leq 1 \quad \forall\, k,$$

$$\sum_{i \in \{2,3,4,5,6\}} \sum_j x_{ijk} \leq 1 \quad \forall\, k.$$

**Restricción 4: Restricción de curso.** Se permite impartir a lo más una asignatura $i$ de cada curso en una misma hora $k$:

$$\sum_{i\in\{0,1,2,3\}} \sum_{j} x_{ijk} \leq 1 \quad \forall\, k,$$
$$\sum_{i\in\{4,5,6,7\}} \sum_{j} x_{ijk} \leq 1 \quad \forall\, k.$$

**Condición de dominio.**

$$x_{ijk} \in \{0,1\} \quad \forall\, i,j,k.$$

**Función objetivo**

Un "horario compacto" busca programar las clases en los primeros bloques de horas posibles. Para incentivar al solucionador a usar los bloques de horas más tempranos, ponderamos la hora $k$ con un costo incremental:

$$\text{mín}\, z = \sum_{i=0}^{7}\sum_{j=0}^{2}\sum_{k=0}^{4}(k+1)\cdot x_{ijk}.$$

### 3.3. Implementación en código

```python
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Planeacion de horario')

# VARIABLES DE DECISION
materias_i = range(8)
aulas_j = range(3)
horas_k = range(5)

x = mdl.binary_var_dict(
    [(i, j, k) for i in materias_i for j in aulas_j for k in horas_k],
    name='x'
)

# RESTRICCIONES
docente_1 = [0, 1, 7]         # M1, M2, M8
docente_2 = [2, 3, 4, 5, 6]   # M3, M4, M5, M6, M7

curso_1 = [0, 1, 2, 3]        # M1, M2, M3, M4
curso_2 = [4, 5, 6, 7]        # M5, M6, M7, M8

# restriccion 1
mdl.add_constraints(
    mdl.sum(x[i, j, k] for j in aulas_j for k in horas_k) == 1
    for i in materias_i
)

# restriccion 2
```

```
mdl.add_constraints(mdl.sum(x[i, j, k] for i in materias_i) <= 1
                    for j in aulas_j
                    for k in horas_k
                    )

# restriccion 3
for k in horas_k:
    mdl.add_constraint(mdl.sum(x[i, j, k] for i in docente_1 for j in
    aulas_j) <= 1)
    mdl.add_constraint(mdl.sum(x[i, j, k] for i in docente_2 for j in
    aulas_j) <= 1)

# restriccion 4
for k in horas_k:
    mdl.add_constraint(mdl.sum(x[i, j, k] for i in curso_1 for j in
    aulas_j) <= 1)
    mdl.add_constraint(mdl.sum(x[i, j, k] for i in curso_2 for j in
    aulas_j) <= 1)

# FUNCION OBJETIVO
mdl.minimize(mdl.sum((k + 1) * x[i, j, k] for i in materias_i for j in
     aulas_j for k in horas_k))

solution = mdl.solve(log_output=True)

if solution:
    print(f"Valor optimo de la funcion objetivo: {solution.
    objective_value}\n")
    print("=================== HORARIO ESCOLAR OPTIMIZADO
    ===================")
    for k in horas_k:
        print(f"\n--- Bloque {k + 1} ---")
        clases_en_bloque = 0
        for j in aulas_j:
            for i in materias_i:
                if x[i, j, k].solution_value > 0.5:
                    print(f"  Aula {j + 1}: Materia M{i + 1}")
                    clases_en_bloque += 1
        if clases_en_bloque == 0:
            print("  (Sin clases programadas)")
```

**Código 5. Planeación de horarios escolares.**

### 3.4. Resultados e interpretación

```

=================== HORARIO ESCOLAR OPTIMIZADO ===================

--- Bloque 1 ---
  Aula 1: Materia M1
  Aula 2: Materia M6

```

```
--- Bloque 2 ---
  Aula 1: Materia M2
  Aula 2: Materia M7

--- Bloque 3 ---
  Aula 1: Materia M3
  Aula 2: Materia M8

--- Bloque 4 ---
  Aula 1: Materia M4

--- Bloque 5 ---
  Aula 1: Materia M5
```

**Código 6. Salida del solver para el Problema 3.**

El modelo logra distribuir las 8 asignaturas utilizando dos de las tres aulas disponibles a lo largo de las 5 horas de la jornada. Al ponderar cada clase por su bloque temporal ($k + 1$), la solución prioriza ocupar los primeros bloques horarios en paralelo (impartiendo dos clases simultáneas en los bloques 1, 2 y 3) y deja únicamente una materia para los bloques 4 y 5.

De igual forma, se verifican las condiciones:

- **Carga docente:** El Docente 1 ($M_1, M_2, M_8$) imparte sus materias en los bloques 1, 2 y 3 respectivamente; el Docente 2 ($M_3, M_4, M_5, M_6, M_7$) las imparte en los bloques 1, 2, 3, 4 y 5. Ningún docente coincide con dos clases en la misma hora.
- **Restricción de cursos:** El Curso 1 ($M_1, M_2, M_3, M_4$) lleva sus materias de manera secuencial en los bloques 1 al 4, mientras que el Curso 2 ($M_5, M_6, M_7, M_8$) lo hace en los bloques 1, 2, 3 y 5. Se cumple que nunca coinciden dos materias de un mismo curso en la misma hora.
- **Uso de espacio:** Cada materia asignada ocupa un aula y hora distintas sin exceder la capacidad máxima de una clase por aula y hora.

# 4. El camino más corto

## 4.1. Datos y supuestos

- Grafo dirigido con $n$ nodos, origen $A$ y destino $N$.
- Cada arco $(i, j)$ tiene asociado un costo $c_{ij}$.

## 4.2. Formulación matemática

### Variables de decisión

Variable binaria $x_{ij}$ que indica si se selecciona el arco $(i, j)$:

$$x_{ij} = \begin{cases} 1 & \text{si se toma el arco de } i \text{ a } j, \\ 0 & \text{si no,} \end{cases}$$

donde $i, j \in \{A, B, C, D, E, F, N\}$.

### Restricciones

**Conservación de flujo.**

$$\sum_j x_{Aj} = 1 \qquad \text{(sale exactamente un arco del origen),}$$

$$\sum_i x_{iN} = 1 \qquad \text{(llega exactamente un arco al destino),}$$

$$\sum_i x_{ik} = \sum_j x_{kj} \quad \forall\, k \in V \setminus \{A, N\} \qquad \text{(conservación en nodos intermedios).}$$

**Condición de dominio.**

$$x_{ij} \in \{0, 1\} \quad \forall\, (i, j) \in \mathcal{A}.$$

### Función objetivo

Minimizar la suma del costo de los arcos seleccionados:

$$\text{mín}\, z = \sum_{(i,j)\in\mathcal{A}} c_{ij} \cdot x_{ij}.$$

## 4.3. Implementación en código

```python
from docplex.mp.model import Model

mdl = Model(name='Camino_Mas_Corto')

# dic de arcos y costos
arcos_costos = {
    ('A', 'B'): 4,
    ('A', 'C'): 2,
```

```
    ('A', 'D'): 5,
    ('B', 'E'): 3,
    ('C', 'B'): 1,
    ('C', 'E'): 6,
    ('C', 'F'): 3,
    ('D', 'F'): 2,
    ('E', 'N'): 6,
    ('F', 'N'): 4
}

# nodos
nodos = ['A', 'B', 'C', 'D', 'E', 'F', 'N']

# vars de decision
x = mdl.binary_var_dict(arcos_costos.keys(), name='x')

# restricciones de flujo
for nodo in nodos:
    flujo_salida = mdl.sum(x[i, j] for i, j in arcos_costos.keys() if
    i == nodo)
    flujo_entrada = mdl.sum(x[i, j] for i, j in arcos_costos.keys() if
     j == nodo)

    if nodo == 'A':
        mdl.add_constraint(flujo_salida - flujo_entrada == 1)
    elif nodo == 'N':
        mdl.add_constraint(flujo_salida - flujo_entrada == -1)
    else:
        mdl.add_constraint(flujo_salida - flujo_entrada == 0)

# funcion objetivo
mdl.minimize(mdl.sum(costo * x[arc] for arc, costo in arcos_costos.
    items()))

# resolver
solution = mdl.solve(log_output=False)

if solution:
    print(f"longitud del camino mas corto: {solution.objective_value}"
    )
    print("arcos usados en el camino:")
    for arc in arcos_costos.keys():
        if x[arc].solution_value > 0.5:
            print(f"  {arc[0]} -> {arc[1]} (Costo: {arcos_costos[arc
    ]})")
```

**Código 7. El camino más corto.**

## 4.4. Resultados e interpretación

**Optimal objective value:** $z = 9$.

```
longitud del camino mas corto: 9.0
arcos usados en el camino:
  A -> C (Costo: 2)
  C -> F (Costo: 3)
  F -> N (Costo: 4)
```

**Código 8. Salida del solver para el Problema 4.**

El camino más corto desde $A$ hasta $N$ es $A \to C \to F \to N$, con una longitud total de 9 unidades.

# 5. Asignación de carteros

## 5.1. Datos y supuestos

- Requerimiento diario de carteros según la demanda de envíos (Tabla 3).
- Cada empleado a tiempo completo trabaja 5 días consecutivos y descansa 2 días.
- Determinar el número mínimo de carteros a tiempo completo.

## 5.2. Formulación matemática

### Variables de decisión

$$x_i = \text{número de trabajadores que inician su turno según el patrón } i,$$

donde $i \in \{0, 1, 2, 3, 4, 5, 6\} = \{\text{LMMiJV}, \text{MMiJVS}, \dots\}$.

**Matriz de cobertura.** Sea $a_{ij} = 1$ si el patrón $i$ cubre el día $j \in \{L, M, Mi, J, V, S, D\}$, y 0 de lo contrario:

$$A = [a_{ij}] = \begin{pmatrix} 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 \\ 1 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 1 \end{pmatrix}.$$

### Restricciones

**Demanda de personal.** Para cada día $j$, la oferta de personal debe ser al menos igual a la demanda $b_j = [17, 13, 15, 19, 14, 16, 11]$:

$$\sum_i x_i a_{ij} \geq b_j \quad \forall\, j.$$

*Nota de modelación:* La igualdad estricta $\sum_i x_i a_{ij} = b_j$ resulta infactible en enteros no negativos, por lo cual se debe usar desigualdad ($\geq$).

### Condición de dominio.

$$x_i \geq 0 \quad \text{entero}, \forall\, i.$$

### Función objetivo

Minimizar el total de trabajadores contratados:

$$\text{mín}\, z = \sum_i x_i.$$

## 5.3. Implementación en código

```python
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Asignacion de carteros')

dias = [0, 1, 2, 3, 4, 5, 6]

# Variable de decision
x = mdl.integer_var_list(len(dias), name='x')  # x_i >= 0 entero

a = [
    [1, 1, 1, 1, 1, 0, 0],
    [0, 1, 1, 1, 1, 1, 0],
    [0, 0, 1, 1, 1, 1, 1],
    [1, 0, 0, 1, 1, 1, 1],
    [1, 1, 0, 0, 1, 1, 1],
    [1, 1, 1, 0, 0, 1, 1],
    [1, 1, 1, 1, 0, 0, 1]
]

a_np = np.array(a)

# Restricciones: demanda
b = [17, 13, 15, 19, 14, 16, 11]

mdl.add_constraints(
        mdl.sum(x[i]*a_np[i][j] for i in dias) >= b[j] for j in range(
    len(b))
)

mdl.minimize(
        mdl.sum(x[i] for i in dias)
        )

solution = mdl.solve(log_output=True,)

print(solution)
```

**Código 9. Asignación de carteros.**

## 5.4. Resultados e interpretación

**Optimal objective value:** $z = 23$.

```
solution for: Asignacion de carteros
objective: 23
status: OPTIMAL_SOLUTION(2)
x_0=6
x_1=4
```

```
x_2=1
x_3=8
x_5=4
```

**Código 10. Salida del solver para el Problema 5.**

Se requiere un total mínimo de 23 carteros para cubrir los requerimientos diarios.

# 6. Distribución de productos

## 6.1. Datos y supuestos

- Plantas de producción: Portland ($P$) y Baltimore ($B$).
- Centros de distribución: Las Vegas ($V$) y Detroit ($D$).
- Costos de envío unitarios, oferta y demanda según las Tablas 4 y 5 del enunciado.
- Minimizar el costo de distribución.

## 6.2. Formulación matemática

### Variables de decisión

Unidades enviadas de la planta $i \in \{P, B\}$ al centro $j \in \{V, D\}$:

$$x_{ij} \geq 0 \quad \text{entero.}$$

### Restricciones

**Restricción de oferta.**

$$\begin{aligned} x_{PV} + x_{PD} &\leq 200 \quad \text{(Portland)}, \\ x_{BV} + x_{BD} &\leq 600 \quad \text{(Baltimore)}. \end{aligned}$$

**Restricción de demanda.**

$$\begin{aligned} x_{PV} + x_{BV} &\geq 300 \quad \text{(Las Vegas)}, \\ x_{PD} + x_{BD} &\geq 400 \quad \text{(Detroit)}. \end{aligned}$$

**Condición de dominio.**

$$x_{ij} \geq 0 \quad \text{entero}, \forall\, i, j.$$

### Función objetivo

$$\text{mín}\, z = 30x_{PV} + 40x_{PD} + 60x_{BV} + 50x_{BD}.$$

## 6.3. Implementación en código

```python
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Distribucion de productos')

productoras = ['P', 'B']
distribuidoras = ['V', 'D']

```

```
x = mdl.integer_var_matrix(productoras, distribuidoras, name='x')

oferta = {'P': 200, 'B': 600}
demanda = {'V': 300, 'D': 400}

# oferta
mdl.add_constraints(
    mdl.sum(x[i, j] for j in distribuidoras) <= oferta[i]
    for i in productoras
)

# demanda
mdl.add_constraints(
    mdl.sum(x[i, j] for i in productoras) >= demanda[j]
    for j in distribuidoras
)

costos = {
    ('P', 'V'): 30, ('P', 'D'): 40,
    ('B', 'V'): 60, ('B', 'D'): 50
}

mdl.minimize(
    mdl.sum(costos[i, j] * x[i, j] for i in productoras for j in
    distribuidoras)
)

solution = mdl.solve(log_output=True,)

print(solution)
```

**Código 11. Distribución de productos.**

## 6.4. Resultados e interpretación

**Optimal objective value:** $z = 32000$.

```
solution for: Distribucion de productos
objective: 32000
status: OPTIMAL_SOLUTION(2)
x_P_V=200
x_B_V=100
x_B_D=400
```

**Código 12. Salida del solver para el Problema 6.**

El plan óptimo envía 200 unidades de Portland a Las Vegas, 100 unidades de Baltimore a Las Vegas y 400 unidades de Baltimore a Detroit, con un costo total de $32,000.

# 7. Contratación de personal

## 7.1. Datos y supuestos

- Planificación de 4 semanas ($t \in \{1, 2, 3, 4\}$).
- Demandas: $D = [52, 65, 70, 85]$.
- 6 expertos iniciales en la semana 1.
- Capacitación: 1 experto capacita hasta 2 novatos.
- Evolución: Novato ($N_t$) $\rightarrow$ Aprendiz ($A_{t+1}$) $\rightarrow$ Experto ($E_{t+2}$).
- Condiciones finales: $N_4 = 0$, $A_4 = 0$ (implica $N_3 = 0$) y $E_4 \geq 9$.

## 7.2. Formulación matemática

### Variables de decisión

Para $t \in \{1, 2, 3, 4\}$: $N_t$ (novatos), $A_t$ (aprendices), $E_t$ (expertos), $E_t^{cap}$ (expertos capacitando), $E_t^{prod}$ (expertos produciendo), $I_t$ (inventario final).

### Restricciones

**Descomposición y capacidad de expertos.**

$$E_t = E_t^{prod} + E_t^{cap}, \quad N_t \leq 2E_t^{cap} \quad \forall\, t.$$

**Equilibrio de personal.**

$$\begin{aligned} &E_1 = 6, \quad A_1 = 0, \\ &A_t = N_{t-1} \quad \forall\, t \in \{2, 3, 4\}, \\ &E_t = E_{t-1} + A_{t-1} \quad \forall\, t \in \{2, 3, 4\}. \end{aligned}$$

**Balance de inventario y producción.**

$$(10E_t^{prod} + 5E_t^{cap} + 5A_t + 1N_t) + I_{t-1} = D_t + I_t \quad \forall\, t \quad (I_0 = 0).$$

**Condiciones de cierre.**

$$N_4 = 0, \quad N_3 = 0, \quad E_4 \geq 9.$$

**Condición de dominio.**

$$N_t, A_t, E_t, E_t^{cap}, E_t^{prod}, I_t \geq 0 \quad \text{enteros}, \forall\, t.$$

### Función objetivo

$$\text{mín}\, z = \sum_{t=1}^{4} (300E_t^{prod} + 400E_t^{cap} + 150A_t + 50N_t).$$

### 7.3. Implementación en código

```
import numpy as np
from docplex.mp.model import Model

mdl = Model(name='Contratacion de personal')

semanas_t = range(4)
demanda = [52, 65, 70, 85]

N = mdl.integer_var_list(semanas_t, lb=0, name='N')
A = mdl.integer_var_list(semanas_t, lb=0, name='A')
E = mdl.integer_var_list(semanas_t, lb=0, name='E')
E_prod = mdl.integer_var_list(semanas_t, lb=0, name='E_prod')
E_cap = mdl.integer_var_list(semanas_t, lb=0, name='E_cap')
I = mdl.integer_var_list(semanas_t, lb=0, name='I')

mdl.add_constraints([E[t] == E_prod[t] + E_cap[t] for t in semanas_t])
mdl.add_constraints([N[t] <= 2 * E_cap[t] for t in semanas_t])

mdl.add_constraint(E[0] == 6)
mdl.add_constraint(A[0] == 0)

mdl.add_constraints([A[t] == N[t - 1] for t in range(1, 4)])
mdl.add_constraints([E[t] == E[t - 1] + A[t - 1] for t in range(1, 4)
    ])

mdl.add_constraint(10 * E_prod[0] + 5 * E_cap[0] + 5 * A[0] + 1 * N[0]
     == demanda[0] + I[0])

mdl.add_constraints([
    (10 * E_prod[t] + 5 * E_cap[t] + 5 * A[t] + 1 * N[t]) + I[t - 1]
    == demanda[t] + I[t]
    for t in range(1, 4)
])

mdl.add_constraint(N[3] == 0)
mdl.add_constraint(N[2] == 0)
mdl.add_constraint(E[3] + A[3] >= 9)

mdl.minimize(
    mdl.sum(300 * E_prod[t] + 400 * E_cap[t] + 150 * A[t] + 50 * N[t]
    for t in semanas_t)
)

solution = mdl.solve(log_output=True)
print(solution)
```

**Código 13. Contratación de personal.**

### 7.4. Resultados e interpretación

**Optimal objective value:** $z = 9200$.

```
solution for: Contratacion de personal
objective: 9200
status: OPTIMAL_SOLUTION(2)
N_0=1, N_1=2, A_1=1, A_2=2, E_0=6, E_1=6, E_2=7, E_3=9
E_prod_0=5, E_prod_1=5, E_prod_2=7, E_prod_3=9, E_cap_0=1, E_cap_1=1
I_0=4, I_1=1, I_2=11, I_3=16
```

**Código 14. Salida del solver para el Problema 7.**

El costo mínimo de contratación y salarios es de $9,200, logrando graduar a los expertos necesarios para cumplir con las condiciones finales.

# 8. Planeación de encuestas

## 8.1. Datos y supuestos

- 1,000 encuestas contratadas.
- Al menos 400 en hogares con mascotas y al menos 400 sin mascotas.
- Total de encuestas en la tarde $\geq$ total en la mañana.
- Al menos 40 % de encuestas con mascotas en la mañana y 60 % en la tarde.
- Costos de la Tabla 7.

## 8.2. Formulación matemática

### Variables de decisión

$x_{ij} \geq 0$ entera, con $i \in \{M, T\}$ (Mañana, Tarde) y $j \in \{C, S\}$ (Con mascotas, Sin mascotas).

### Restricciones

$$\sum_i \sum_j x_{ij} = 1000 \quad \text{(total)},$$

$$\sum_i x_{iC} \geq 400, \quad \sum_i x_{iS} \geq 400 \quad \text{(mínimos por tipo)},$$

$$\sum_j x_{Tj} \geq \sum_j x_{Mj} \quad \text{(tarde} \geq \text{mañana)},$$

$$x_{MC} \geq 0{,}4 \sum_i x_{iC}, \quad x_{TC} \geq 0{,}6 \sum_i x_{iC}.$$

### Condición de dominio.

$$x_{ij} \geq 0 \quad \text{entero}, \forall\, i, j.$$

### Función objetivo

$$\text{mín}\, z = 20x_{MC} + 18x_{MS} + 25x_{TC} + 20x_{TS}.$$

## 8.3. Implementación en código

```
from docplex.mp.model import Model
import numpy as np

mdl = Model(name='Planeacion de encuestas')

costos = [
    [20, 25],  # Con mascotas (M, T)
```

```
    [18, 20],  # Sin mascotas (M, T)
]
costos_np = np.array(costos)

tipos_i = range(2)        # 0: Con mascotas, 1: Sin mascotas
horarios_j = range(2) # 0: Manana, 1: Tarde

x = mdl.integer_var_matrix(tipos_i, horarios_j, lb=0, name='x')

mdl.add_constraint(mdl.sum(x[i,j] for i in tipos_i for j in horarios_j
    ) == 1000)
mdl.add_constraints(mdl.sum(x[i,j] for j in horarios_j) >= 400 for i
    in tipos_i)
mdl.add_constraint(mdl.sum(x[i,1] for i in tipos_i) >= mdl.sum(x[i,0]
    for i in tipos_i))
mdl.add_constraint(x[0,0] >= 0.4 * mdl.sum(x[0,j] for j in horarios_j)
    )
mdl.add_constraint(x[0,1] >= 0.6 * mdl.sum(x[0,j] for j in horarios_j)
    )

mdl.minimize(mdl.sum(costos_np[i,j] * x[i,j] for i in tipos_i for j in
     horarios_j))

solution = mdl.solve(log_output=True)
print(solution)
```

**Código 15. Planeación de encuestas.**

## 8.4. Resultados e interpretación

**Optimal objective value:** $z = 20520$.

```
solution for: Planeacion de encuestas
objective: 20520
status: OPTIMAL_SOLUTION(2)
x_0_0=160
x_0_1=240
x_1_0=340
x_1_1=260
```

**Código 16. Salida del solver para el Problema 8.**

El costo mínimo total es de $20,520 con 160 encuestas con mascotas por la mañana, 240 con mascotas por la tarde, 340 sin mascotas por la mañana y 260 sin mascotas por la tarde.

# 9. Planeación de transporte

## 9.1. Datos y supuestos

- Oferta en plantas $A = 8, B = 5, C = 3$.
- Demanda en almacenes $P = 5, Q = 4, S = 3$.
- Costos expresados en la Tabla 8 del enunciado.

## 9.2. Formulación matemática

### Variables de decisión

$x_{ij} \geq 0$ entera, número de máquinas a transportar de la planta $i \in \{A, B, C\}$ al almacén $j \in \{P, Q, S\}$.

### Restricciones

**Demanda y oferta.**

$$\sum_i x_{iP} = 5, \quad \sum_i x_{iQ} = 4, \quad \sum_i x_{iS} = 3,$$
$$\sum_j x_{Aj} \leq 8, \quad \sum_j x_{Bj} \leq 5, \quad \sum_j x_{Cj} \leq 3.$$

**Condición de dominio.**

$$x_{ij} \geq 0 \quad \text{entero}, \forall\, i, j.$$

### Función objetivo

$$\text{mín}\, z = 50x_{AP} + 60x_{AQ} + 30x_{AS} + 60x_{BP} + 40x_{BQ} + 20x_{BS} + 40x_{CP} + 70x_{CQ} + 30x_{CS}.$$

## 9.3. Implementación en código

```
from docplex.mp.model import Model
import numpy as np

costos = [
    [50, 60, 30],  # A
    [60, 40, 20],  # B
    [40, 70, 30]   # C
]
costos_np = np.array(costos)

mdl = Model(name='Planeacion de transporte')

plantas_i = ['A', 'B', 'C']
almacenes_j = ['P', 'Q', 'S']
```

```
15
16 x = mdl.integer_var_matrix(plantas_i, almacenes_j, lb=0, name='x')
17
18 mdl.add_constraint(mdl.sum(x[i, 'P'] for i in plantas_i) == 5)
19 mdl.add_constraint(mdl.sum(x[i, 'Q'] for i in plantas_i) == 4)
20 mdl.add_constraint(mdl.sum(x[i, 'S'] for i in plantas_i) == 3)
21
22 mdl.add_constraint(mdl.sum(x['A', j] for j in almacenes_j) <= 8)
23 mdl.add_constraint(mdl.sum(x['B', j] for j in almacenes_j) <= 5)
24 mdl.add_constraint(mdl.sum(x['C', j] for j in almacenes_j) <= 3)
25
26 mdl.minimize(
27     mdl.sum(costos_np[i_idx, j_idx] * x[i, j]
28             for i_idx, i in enumerate(plantas_i)
29             for j_idx, j in enumerate(almacenes_j))
30 )
31
32 solution = mdl.solve(log_output=True)
33 print(solution)
```

**Código 17. Planeación de transporte.**

## 9.4. Resultados e interpretación

**Optimal objective value:** $z = 460$.

```
1 solution for: Planeacion de transporte
2 objective: 460
3 status: OPTIMAL_SOLUTION(2)
4 x_A_P=2, x_A_S=2, x_B_Q=4, x_B_S=1, x_C_P=3
```

**Código 18. Salida del solver para el Problema 9.**

El costo mínimo de transporte es de $460.

# 10. Personal de obra

## 10.1. Datos y supuestos

- Requerimientos por intervalo de 4 horas según la Tabla 9.
- Turnos de PQR de 8 horas continuas.
- El salario se duplica en el horario de 00:00 a 06:00 h.

## 10.2. Formulación matemática

### Inciso a): Modelo base con personal PQR

Variables $x_i \geq 0$ entera, número de trabajadores PQR que inician turno en el periodo $i \in \{1, \ldots, 6\}$.

$$\begin{aligned} x_6 + x_1 &\geq 70 \quad (06-10), \\ x_1 + x_2 &\geq 20 \quad (10-14), \\ x_2 + x_3 &\geq 80 \quad (14-18), \\ x_3 + x_4 &\geq 30 \quad (18-22), \\ x_4 + x_5 &\geq 10 \quad (22-02), \\ x_5 + x_6 &\geq 10 \quad (02-06). \end{aligned}$$

$$\text{mín } z = 8(x_1 + x_2 + x_3) + 10x_4 + 14x_5 + 12x_6.$$

### Inciso b): Inclusión de TrabajoFlex

Agregando variables $y_i \geq 0$ entera para personal de TrabajoFlex contratado por periodos de 4 horas con costo de 14,4 unidades ($1{,}8 \times 8$):

$$\text{mín } z = 8(x_1 + x_2 + x_3) + 10x_4 + 14x_5 + 12x_6 + 14{,}4 \sum_{i=1}^{6} y_i.$$

## 10.3. Implementación en código

```
## ==================== ##
# ------ MODELO
## ==================== ##
import numpy as np
from docplex.mp.model import Model

mdl = Model(name='Personal de obra')

def personal_obra(mdl, outsorcing=False):
    ## ======================================== ##
    # ------ Formulacion matematica
    ## ======================================== ##

    # Variables de decision
    turnos_i = [1, 2, 3, 4, 5, 6]
```

```
16      turnos_i = np.linspace(1, 6, 6, dtype=int)
17
18      x = mdl.integer_var_list(turnos_i, lb=0, name='x')
19
20      if outsorcing:
21          y = mdl.integer_var_list(turnos_i, lb=0, name='y')
22
23      ## ========================================== ##
24      # ------ Restricciones
25      ## ========================================== ##
26      if outsorcing:
27          mdl.add_constraints([
28              x[5] + x[0] + y[0] >= 70,  # Turno 6 + Turno 1
29              x[0] + x[1] + y[1] >= 20,  # Turno 1 + Turno 2
30              x[1] + x[2] + y[2] >= 80,  # Turno 2 + Turno 3
31              x[2] + x[3] + y[3] >= 30,  # Turno 3 + Turno 4
32              x[3] + x[4] + y[4] >= 10,  # Turno 4 + Turno 5
33              x[4] + x[5] + y[5] >= 10   # Turno 5 + Turno 6
34          ])
35      else:
36          mdl.add_constraints([
37              x[5] + x[0] >= 70,  # Turno 6 + Turno 1
38              x[0] + x[1] >= 20,  # Turno 1 + Turno 2
39              x[1] + x[2] >= 80,  # Turno 2 + Turno 3
40              x[2] + x[3] >= 30,  # Turno 3 + Turno 4
41              x[3] + x[4] >= 10,  # Turno 4 + Turno 5
42              x[4] + x[5] >= 10   # Turno 5 + Turno 6
43          ])
44
45      ## ========================================== ##
46      # ------ Funcion Objetivo
47      ## ========================================== ##
48      # Funcion objetivo
49      if outsorcing:
50          mdl.minimize(
51              mdl.sum( 8*(x[0] + x[1] + x[2])+ 10*x[3] + 14*x[4] + 12*x
    [5]
52                       + 14.4*mdl.sum(y[i-1] for i in turnos_i) )
53          )
54      else:
55          mdl.minimize(
56              mdl.sum( 8*(x[0] + x[1] + x[2])+ 10*x[3] + 14*x[4] + 12*x
    [5] )
57          )
58
59      # Resolver e imprimir solucion
60      solution = mdl.solve(log_output=True)
61
62      return solution
63
64
65  if __name__ == "__main__":
66      solution = personal_obra(mdl, outsorcing=True)
67      print(solution)
```

**Código 19. Personal de obra.**

### 10.4. Resultados e interpretación

**Optimal objective value:** $z = 1340$.

```
solution for: Personal de obra
objective: 1340
status: OPTIMAL_SOLUTION(2)
x_1=60
x_3=80
x_4=10
x_6=10
```

**Código 20. Salida del solver para el Problema 10.**

El valor óptimo de la función objetivo alcanzado por el solver es $z = 1340$, lo cual representa el costo salarial total mínimo para cumplir con los requerimientos diarios de personal.

- **Inciso a) Plan óptimo de contratación de PQR:** La asignación óptima requiere contratar un total de 160 empleados a tiempo completo repartidos en 4 turnos:
  - 60 trabajadores iniciando en el periodo 2 / $x_1$ (10:00–18:00 h).
  - 80 trabajadores iniciando en el periodo 4 / $x_3$ (18:00–02:00 h).
  - 10 trabajadores iniciando en el periodo 5 / $x_4$ (22:00–06:00 h).
  - 10 trabajadores iniciando en el periodo 1 / $x_6$ (02:00–10:00 h).
- **Inciso b) Análisis de la conveniencia de TrabajoFlex:** El solver asigna $y_i = 0$ para todos los periodos $i \in \{1, \ldots, 6\}$. Esto indica que a PQR **no le conviene contratar personal de TrabajoFlex**.

## Material de la tarea

*Problemas de Programación Binaria y Entera*. Métodos de Optimización I, CIMAT Aguascalientes.